

SIMULACIÓN DE MUNDO CON **PROGRAMACIÓN CONCURRENTE**

Rubén Pavón Rollán

1. Introducción	3
1.1 Motivación del proyecto	3
1.2 Objetivos principales	3
1.3 Requisitos de la asignatura cubiertos	3
Arquitectura del Sistema Concurrente	4
2.1 Modelo de actores e infraestructura multiproceso	4
2.2 Hilos de control internos (multithreading)	5
2.3 Primitivas de sincronización temporal	5
2.4 Exclusión mutua reentrante y prevención de deadlocks	5
3. Protocolo de comunicación dinámica e inyección de caos	6
3.1 Estructura del mensaje transaccional	6
3.2 Fiabilidad en redes asíncronas	6
3.3 Tolerancia a fallos y desconexión dinámica de nodos muertos	7
3.4 Optimización de red y gestión de relaciones mediante algoritmo Clock	7
3.5 Modelado del caos y estocasticidad	8
4. Diseño de módulos y modelos matemáticos	8
4.1 Módulo Demográfico: modelo matricial de Leslie	8
4.2 Módulo Económico: función de producción de Cobb-Douglas	9
4.3 Módulo Agrícola: modelo ecológico World3	10
4.4 Módulo de Salud: modelo epidemiológico SIR	10
4.5 Módulo Sociopolítico: ecuación logística de inestabilidad	10
5. Manual de configuración y ejecución	11
5.1 Arquitectura de la interfaz gráfica (GUI)	11
5.2 Orquestación del despliegue	11
6. Pruebas y calibración	12
6.2 Pruebas de estrés y horizontes temporales	12
6.2 Experimentación y calibración de condiciones iniciales	12
6.3 Ajuste de las fórmulas matemáticas	13
7. Declaración de IA y aspectos técnicos	13
7.1 Declaración de uso de Inteligencia Artificial	13
7.2 Guía de mantenimiento	13
7.3 Limitaciones técnicas y comportamiento inerte de la GUI	14
8. Conclusiones	15
9. Bibliografía	15

1. Introducción

1.1 Motivación del proyecto

El estudio y modelado de sistemas globales dinámicos ha sido históricamente un reto computacional. Simular un desarrollo del “*mundo real*” compuesto por entidades autónomas que interactúan, compiten y cooperan de manera asíncrona a través de canales de comunicación imperfectos.

La motivación principal de este proyecto es el desarrollo de un simulador global interactivo, donde cada continente actúa como un sistema completamente aislado en un entorno autónomo. El proyecto no se centra solamente en la fidelidad de las ecuaciones macroeconómicas, sino en cómo el caos orgánico, la volatilidad financiera y las crisis sanitarias emergen de forma natural a partir de la concurrencia y la inestabilidad de las comunicaciones.

1.2 Objetivos principales

Para cumplir las necesidades del proyecto se fijaron los siguientes objetivos técnicos y funcionalidades:

- **Aislamiento de nodos mediante multiprocesamiento:** diseñar una arquitectura donde cada civilización se ejecute en su propio espacio de memoria virtual direccionable, forzando a que cualquier interacción se realice exclusivamente mediante el intercambio de información y eliminando el uso de variables globales compartidas.
- **Diseño de un protocolo de comunicación fiable y asíncrono:** desarrollar un protocolo de comunicación por paso de mensajes que gestione transacciones económicas, sin bloquear los hilos de ejecución principales, garantizando la consistencia del capital en vuelo.
- **Inyección de estocasticidad y modelado del caos:** introducir perturbaciones dinámicas en la red, como la pérdida de paquetes o la propagación de enfermedades a través de rutas comerciales asociativas, para romper el equilibrio matemático lineal y evaluar la resiliencia del ecosistema global.
- **Desarrollo de una interfaz gráfica reactiva:** implementar un panel de control y un *dashboard* interactivo que permita parametrizar la simulación antes de su lanzamiento y analizar los resultados de forma visual e intuitiva.

1.3 Requisitos de la asignatura cubiertos

Este desarrollo ha aplicado de forma práctica los fundamentos de la asignatura, cumpliendo con los requisitos establecidos para el proyecto. A continuación se justifican los requisitos cubiertos en la implementación:

- **Arquitectura multiproceso (*multiprocessing*)**: cada continente hereda de la clase base de procesos de *Python*, abstrayendo el comportamiento de un sistema distribuido donde los nodos carecen de memoria compartida.
- **Programación multihilo (*multithreading*)**: dentro de cada proceso se explota la concurrencia interna separando el motor matemático (hilo principal) de las tareas de red y mensajería (hilo de escucha asíncrono), optimizando los tiempos de cómputo.
- **Paso de mensajería mediante colas asíncronas (*Queue*)**: se implementa una topología de red descentralizada, donde los buzones de comunicación actúan como canales de paso de mensajes emulando la latencia y el comportamiento de los *sockets* de red.
- **Exclusión mutua reentrante (*RLock*)**: se asegura la integridad del estado local de cada civilización frente a accesos concurrentes mediante cerrojos reentrantes, previniendo *race conditions* y *deadlocks*.
- **Sincronización temporal por barreras**: se coordina el avance de los ciclos mediante primitivas de sincronización colectiva (*multiprocessing.barrier*), asegurando que ningún nodo adelante temporalmente al resto y permitiendo un registro de datos consistente para su posterior análisis.

2. Arquitectura del Sistema Concurrente

2.1 Modelo de actores e infraestructura multiproceso

Para emular de manera realista un sistema distribuido, se ha adoptado por una aproximación basada en el *Modelo de Actores*. En este caso, cada civilización se concibe como una entidad autónoma y aislada que posee control total sobre su estado interno y que no comparte memoria con ningún otro sistema. Esto se ha conseguido mediante el uso del módulo *multiprocessing* de *Python*, haciendo que la clase *Civilización* herede directamente de *multiprocessing.Process*.

- **Aislamiento de memoria**: al arrancar cada proceso con el método *start()*, el sistema operativo genera un espacio de direccionamiento virtual propio para cada continente. Esto garantiza la imposibilidad de que una civilización altere las variables económicas o demográficas de otra de forma directa, forzando que toda interacción se realice estrictamente mediante el intercambio de mensajes.
- **Topología de red descentralizada**: en lugar de depender de un servidor centralizado o de un orquestador de comunicaciones, los procesos se configuran en una topología *P2P* en local^[1]. Al inicializarse, cada proceso recibe un diccionario asociativo (*directorio*), que contiene los nombres de las civilizaciones como *keys*, y como valor contiene las referencias a los buzones de entrada correspondientes a cada civilización. De este modo, cualquier nodo conoce los canales para comunicarse con el resto de forma directa.

[1] Aunque la simulación se ejecute en local en un único dispositivo, el sistema implementa una arquitectura *P2P* mediante a nivel lógico. El aislamiento estricto de los procesos fuerza una topología descentralizada, donde la comunicación se debe hacer estrictamente con paso de mensajes.

2.2 Hilos de control internos (*multithreading*)

El rendimiento y la asincronía de cada nodo se optimizan mediante el uso de concurrencia interna con hilos. Cada proceso de civilización divide su ejecución en dos subtarefas con diferentes responsabilidades:

- **Hilo principal (motor matemático):** es el encargado de ejecutar de forma secuencial el método `_motor()`. Este hilo computa las ecuaciones macroeconómicas, la transición demográfica, la gestión sanitaria local y las decisiones comerciales o de contención epidemiológica.
- **Hilo secundario (*_correo*):** diseñado como un hilo *daemon*, corre en segundo plano y permanece bloqueado en una escucha activa sobre su propio buzón mediante llamadas síncronas a `self.buzon.get()`. Su función exclusiva es actuar como interfaz de red: recibe paquetes entrantes de otras civilizaciones, los procesa, y actualiza el estado del nodo de forma asíncrona.

2.3 Primitivas de sincronización temporal

Aunque, la comunicación entre civilizaciones es puramente asíncrona, la simulación requiere una coherencia temporal global para que los datos históricos y las estadísticas agregadas mantengan el sentido cronológico. Para resolver este problema de coordinación sin recurrir a una arquitectura centralizada, se hace uso de una barrera de sincronización distribuida, configurada con un umbral de paso igual al número total de procesos activos. El flujo de ejecución con esta barrera sigue el siguiente protocolo en el método `run()`:

1. El hilo principal computa el año actual en el `motor()`
2. Registra los datos en el búfer de datos mediante `_registrar()`
3. Invoca a la barrera (`self.barrera.wait()`). En este punto, el hilo principal se bloquea cediendo el control del procesador.
4. Una vez el último proceso invoca la barrera, esta se libera, permitiendo que los procesos comiencen un nuevo ciclo.

2.4 Exclusión mutua reentrante y prevención de deadlocks

La coexistencia del hilo principal y el hilo de correo dentro del mismo espacio de memoria introduce el riesgo de condiciones de carrera. Para garantizar la exclusión mutua, se protege el acceso a los módulos internos de la civilización utilizando cerrojos. Sin embargo, el diseño del protocolo comercial introdujo un desafío de concurrencia algo complejo: al final del ciclo económico, el hilo principal adquiere el cerrojo para evaluar las exportaciones en `_comercio()`, y desde ese bloque protegido invoca a `_actualizarCache()`, que a su vez requiere solicitar el cerrojo para inspeccionar los bits de control. Si se emplease un cerrojo estándar (`threading.Lock`), el hilo principal sufriría un bloqueo mutuo consigo mismo, congelando el proceso y provocando que el resto de civilizaciones se queden esperando a una barrera que nunca se va a abrir.

Para solucionar este problema, la arquitectura implementa un cerrojo reentrante ([`threading.RLock`](#)). El `RLock` identifica qué hilo específico posee el control del cerrojo. Si el hilo principal ya ha bloqueado el estado en el motor, el `RLock` le permite el acceso a los métodos

internos de control de caché sin autofrenarse, incrementando un contador de adquisiciones interno. El cerrojo solo se libera eficazmente cuando el hilo principal sale por completo del nivel superior de anidamiento, garantizando una exclusión mutua segura y eficiente.

3. Protocolo de comunicación dinámica e inyección de caos

3.1 Estructura del mensaje transaccional

Para estructurar el intercambio de información entre los nodos distribuidos de forma homogénea, se ha definido una capa de abstracción mediante la clase encapsulada *Mensaje*. Cada paquete de datos que circula por las colas del sistema se instancia bajo este modelo, que cuenta con los siguientes atributos:

- **Identificador único (*id*):** una cadena alfanumérica generadas mediante la especificación UUID (*uuid.uuid4*), necesario para el rastreo de operaciones financieras a través de la red.
- **Campos de enrutamiento (*origen, destino*):** cadenas que identifican formalmente el proceso emisor y el proceso receptor del paquete
- **Control de tipado (*tipo*):** una etiqueta que determina la semántica del mensaje, permitiendo una clasificación eficiente a la hora de desempaquetarlo en el destino.
- **Carga útil (*payload*):** un diccionario flexible que encapsula las variables numéricas y los datos específicos asociados a la operación.

3.2 Fiabilidad en redes asíncronas

El comercio se ha diseñado bajo las premisas de los sistemas tradicionales distribuidos. Originalmente, el intercambio de recursos se ejecutaba de manera directa y ciega, lo que alteraba artificialmente los balances económicos locales. Para dotar al sistema de realismo y consistencia, se ha implementado, de manera simple, un protocolo de acuses de recibo asíncrono (*2-way-handshake*), el cual se ejecuta en tres fases:

1. **Fase de expedición y deducción del inventario:** el hilo principal de la civilización emisora evalúa la viabilidad del contrato comercial. Si dispone de excedentes, deduce los recursos correspondientes de sus almacenes locales y descuenta temporalmente el coste logístico de su PIB. Acto seguido, registra el derecho de cobro futuro en las ventas pendientes vinculándolo al *id* del mensaje enviado.
2. **Fase de recepción y generación del ACK:** el mensaje de tipo "*comercio*" es extraído de la cola por el hilo *_correo* del nodo receptor. Este añade las materias primas a sus módulos de almacenamiento internos y, de forma inmediata, genera una confirmación de pago de tipo "*ack_comercio*". En la carga útil de esta respuesta se introduce el *id_referencia* del mensaje original, remitiendo el paquete de vuelta al emisor.
3. **Fase de cobro:** al recibir el paquete con el ACK, el hilo de correo de la civilización vendedora recupera y elimina la transacción de las ventas pendientes, utilizando el

identificador de referencia. Finalmente, se consolida el dinero real añadiendo el pago en su PIB mediante el método correspondiente.

3.3 Tolerancia a fallos y desconexión dinámica de nodos muertos

En los sistemas distribuidos reales, los nodos pueden sufrir caídas imprevistas o apagados repentinos. En el simulador, esto ocurre cuando la población de un continente decae hasta alcanzar un valor nulo, forzando la muerte del proceso. Si el resto de nodos continuara operando ignorando el deceso, seguirían seleccionando al nodo caído como socio comercial viable, enviando flujos de recursos y capitales logísticos a una cola huérfana que jamás llegaría a procesar los mensajes ni retornaría los ACKs. Esto induciría un colapso financiero global por fuga de capitales en tránsito.

Para dotar a la arquitectura de tolerancia a fallos, se ha implementado un mecanismo de reconfiguración dinámica de la topología. Cuando una civilización colapsa, su hilo principal introduce el ciclo del motor y salta al bloque *finally*. Desde este bloque, antes de liberar sus recursos del sistema operativo, el proceso genera un mensaje de difusión que anuncia su defunción. Al recibir este mensaje, los hilos de *_correo* de los continentes supervivientes ejecutan de forma segura una exclusión mutua para eliminar la llave de la civilización difunta de su propio directorio. A partir de este instante, la topología de la red se contrae, impidiendo que el motor matemático de los nodos vivos vuelva a seleccionar al canal inactivo en sus rutinas comerciales.

3.4 Optimización de red y gestión de relaciones mediante algoritmo Clock

Para evitar que los vectores de contagio epidemiológico se propaguen uniformemente por todo el planeta de manera irreal, se ha diseñado una caché de relaciones comerciales activas gobernada por el algoritmo de *Second Chance*.

Cada civilización cuenta con una estructura *self.cache_comercio* limitada a una capacidad variable de elementos (2 por defecto). Las interacciones modifican el estado de la caché en el método *actualizar Cache(socio)* siguiendo estas reglas:

- **Acierto (hit):** si el socio comercial se encuentra registrado, se interpreta como actividad reciente. El sistema actualiza su bit de control poniéndolo a 1, lo que le otorga una “segunda oportunidad” frente a futuras expulsiones.
- **Inserción directa:** si el socio no está en la caché y hay espacio libre, se introduce directamente asignándole un bit de control con el valor 1.
- **Fallo de caché y reemplazo (miss):** si la caché está saturada y llega un nuevo socio que no está registrado, se ejecuta una inspección lineal sobre las llaves del diccionario. A aquellos socios con bit de control a 1 se les degrada estableciendo su bit de control a 0. El primer socio que tenga el bit de control igual a 0 es inmediatamente expulsado de la caché mediante un borrado físico, cediendo su posición al nuevo nodo comercial, que ingresa con su bit en 1.

Esta implementación permite que, si se deseara implementar una forma de almacenar información de civilizaciones vecinas de manera local y acceder a ellas, solo sea necesario cambiar la información que se guarda en la caché. De esta forma, no tendría que preguntarse por los datos guardados cada vez que los necesite, solo mirar en la caché.

3.5 Modelado del caos y estocasticidad

La aleatoriedad que rompe el determinismo lineal del simulador se concentra en el método `_enviarMensaje()`. Se ha programado una tasa de pérdida de paquetes del 15% mediante una evaluación estocástica (`random.random() > 0.15`).

Si el factor de caos determina la pérdida de la trama, el mensaje es descartado antes de ser inyectado en la cola del destinatario. Este descarte de bajo nivel provoca que el protocolo *2-way handshake* quede inconcluso, obligando a los continentes a asimilar pérdidas logísticas reales que perturban de forma impredecible sus ecuaciones macroeconómicas.

Asimismo, en el ámbito sanitario, el caos se propaga dinámicamente: las civilizaciones sólo evalúan el contagio por pandemia de aquellos continentes que se encuentran en ese ciclo exacto dentro de su caché de comercio. Así, la enfermedad no se difunde de manera uniforme, sino que aprovecha las relaciones comerciales, generando olas epidemiológicas asimétricas, caóticas y altamente diferenciadas entre ejecuciones.

4. Diseño de módulos y modelos matemáticos

4.1 Módulo Demográfico: modelo matricial de Leslie

La dinámica poblacional de cada civilización no se ha modelado mediante un crecimiento exponencial simple, sino empleando una estructura de edad compartimentada basada en una variación discreta de la *Matriz de Leslie*. La población de cada proceso se divide en tres cohortes generacionales:

$$\mathbf{P}(t) = \begin{bmatrix} P_{\text{jovenes}}(t) \\ P_{\text{activos}}(t) \\ P_{\text{ancianos}}(t) \end{bmatrix}$$

El avance temporal y la transición biológica entre los grupos de edad se computan de forma anual multiplicando el vector de estado por una matriz de proyección dinámica L en el método:

$$P(t + 1) = L \times P(t)$$

La matriz de transición se configura dinámicamente cada año en función de las variables del entorno, quedando definida como:

$$L = \begin{bmatrix} (1 - t_1) \cdot s_{base} & f_2 & 0 \\ t_1 \cdot s_{base} & (1 - t_2) \cdot s_{base} & 0 \\ 0 & t_2 \cdot s_{base} & 0.85 \cdot s_{base} \end{bmatrix}$$

Donde los parámetros responden a las siguientes funciones matemáticas:

- **Tasa de transacción (t_1, t_2):** constantes fijadas en que determinan la velocidad nominal con la que los individuos promocionan de cohorte.
- **Factor de supervivencia (s_{base}):** depende de la forma directa del índice de salud del continente y de la seguridad alimentaria:

$$s_{base} = \min(0.98, 0.90 + \frac{salud}{1000}) * \min(1.0, seguridad_alimentaria)$$

- **Fecundidad mitigada (f_2):** representa la tasa de natalidad de la cohorte activa. Parte de un valor máximo ($f_{max} = 0.25$) que sufre una degradación multiplicativa por factores socioeducativos.

4.2 Módulo Económico: función de producción de Cobb-Douglas

El PIB anual de cada continente se calcula mediante el motor macroeconómico basado en la función de producción neoclásica de Cobb-Douglas con rendimientos constantes a escala:

$$Y = A \cdot E \cdot F_{toxico} \cdot K^\alpha \cdot L^{1-\alpha}$$

Donde los componentes del sistema se corresponden con las siguientes variables:

- **Y:** el PIB resultante
- **A:** el nivel tecnológico de la civilización
- **K:** el capital físico efectivo, el cual sufre una tasa de depreciación variable
- **L:** la fuerza laboral activa, extraída directamente de la cohorte central del vector demográfico
- **α :** parámetro de elasticidad de producción fijado en 0.65, priorizando la intensidad del capital sobre la mano de obra
- **E:** representa la eficiencia energética, una penalización logística que deprime el aparato industrial si colapsa el suministro energético
- **F_{toxico} :** una función exponencial decreciente que modela las externalidades negativas de la polución sobre el rendimiento de las infraestructuras

4.3 Módulo Agrícola: modelo ecológico World3

La producción de alimentos se ha diseñado adaptando las dinámicas de límites al crecimiento del modelo World3 de Meadows. La capacidad biológica del suelo no es estática, sino que fluctúa debido a bucles de retroalimentación.

$$Produccion = \min(L \cdot 30.0, T_{arable} \cdot 15.0 \cdot fertilidad \cdot tecnologia \cdot intensidad \cdot e^{-0.0001 \cdot contaminación})$$

La variable *intensidad* encapsula las inversiones de capital químico y energético. Cada año, la fertilidad del suelo decae de forma directa proporcional a la presión demográfica local e incrementa mediante la inversión estatal de capital de regeneración ecológica.

4.4 Módulo de Salud: modelo epidemiológico SIR

La propagación de agentes patógenos dentro de un continente sigue la formulación matemática de un modelo *Susceptibles – Infectados – Recuperados (SIR)* discretizado por años:

$$\Delta I = S \cdot \left(\frac{I}{N}\right) \cdot \beta_{mitigada}$$

- **Tasa de contagio ($\beta_{mitigada}$):** se calcula reduciendo exponencialmente la inefectividad base en función de los avances médicos y de infraestructura tecnológica del nodo infectado:

$$\beta_{mitigada} = \beta_{base} \cdot e^{-0.2 \cdot tecnologia}$$

- **Saturación hospitalaria:** el sistema monitoriza de forma crítica el volumen de infectados frente a la infraestructura sanitaria. Si el volumen de infectados desborda este umbral, la tasa de letalidad de la enfermedad sufre una transición crítica, pasando de un 2% nominal en camas de aislamiento a un 15% de letalidad por colapso clínico sobre la población excedente.

4.5 Módulo Sociopolítico: ecuación logística de inestabilidad

Las revoluciones y alzamientos civiles se gobiernan mediante una función de distribución de probabilidad logística que evalúa el descontento de las masas. La probabilidad de que estalle una revuelta violenta en un ciclo se modela mediante las siguientes ecuaciones:

$$z = -4.0 + 5.0 \cdot Gini + 10.0 \cdot Racionamiento - 2.0 \cdot PIB_{per_capita}$$
$$P(Revolucion) = \frac{1}{1 + e^{-z}}$$

Esta formulación matemática garantiza que el riesgo sociopolítico crezca exponencialmente a medida que aumenta la desigualdad económica (*Gini*) y las tasas de desnutrición por racionamiento alimentario, al tiempo que es amortiguado por la riqueza per cápita del sistema. Si el evento estocástico determina el inicio de la revolución, se aplica una penalización demográfica, eliminando un porcentaje de la población repartido de forma asimétrica.

5. Manual de configuración y ejecución

5.1 Arquitectura de la interfaz gráfica (GUI)

Para facilitar la interacción con el simulador sin necesidad de manipular ficheros, se ha desarrollado una interfaz gráfica nativa utilizando la librería *Flet*. La interfaz se estructura bajo un patrón de diseño desacoplado en tres pantallas independientes y secuenciales:

- **Pantalla de parametrización:** consiste en un panel dividido en cuadrículas que permite configurar las condiciones iniciales de cada continente. El componente cuenta con un validador de tipos por excepciones que sustituye de forma automática cualquier entrada no válida por los valores por defecto del modelo antes de su persistencia.
- **Pantalla de carga:** actúa como nexo de unión entre el hilo de renderizado de la GUI y el motor de simulación. Inicializa un canal de comunicación asíncrono y delega la ejecución de los procesos hijos a un hilo secundario para evitar que la interfaz de usuario se congele debido a la carga computacional del multiprocesamiento.
- **Dashboard:** una vez concluida la ejecución de los ciclos anuales establecidos, esta pantalla lee y unifica los ficheros históricos CSV. Permite al usuario filtrar dinámicamente qué combinaciones de continentes y variables macroeconómicas o sanitarias desea inspeccionar, abstrayendo la lógica de visualización mediante componentes interactivos.

5.2 Orquestación del despliegue

Debido a las restricciones de arquitectura que imponen los diferentes sistemas operativos sobre la clonación de hilos y memoria compartida, el despliegue requiere de una configuración de entorno estricta para garantizar que los procesos hijos se inicialicen de forma homogénea. El simulador se ha dotado de *scripts* de automatización nativos para asegurar su portabilidad tanto en entornos Windows como en sistemas UNIX (Linux/macOS).

En ambos sistemas el arranque se gestiona mediante un archivo ejecutable (.bat para Windows y .sh para Linux/macOS). El archivo ejecuta un *script* que monta el entorno virtual, instalando las librerías necesarias para la ejecución de la simulación. Una vez instaladas, despliega automáticamente la interfaz de configuración para que el usuario pueda comenzar la simulación.

6. Pruebas y calibración

6.2 Pruebas de estrés y horizontes temporales

Con el objetivo de validar la estabilidad de la arquitectura concurrente y la convergencia de los modelos matemáticos a largo plazo, se probaron cuatro escenarios. Estas pruebas permitieron constatar que el intercambio de mensajes a través de colas de comunicación y la sincronización por barreras operaban de forma correcta bajo diferentes cargas de trabajo:

- **Escenario 1 (10 ciclos):** diseñado principalmente para la depuración inicial del protocolo *2-way handshake* comercial, la verificación del enrutamiento de paquetes en el directorio y la validación de la persistencia inmediata en el buffer de los archivos CSV.
- **Escenario 2 (100 ciclos):** utilizado como base de calibración para evaluar la estabilidad de las diferentes variables del ecosistema, antes de introducir desajustes severos en la simulación.
- **Escenario 3 (1000 ciclos):** prueba de escala avanzada orientada a analizar el comportamiento asintótico del sistema. Esta prueba permitió observar la transición ecológica de la contaminación, la estabilización de los virus en estados endémicos con comportamientos estocásticos de alta frecuencia y el comportamiento del capital *en vuelo* ante fluctuaciones severas del mercado.
- **Escenario 4 (10_000 ciclos):** ejecutado con el propósito de observar si, a muy largo plazo existían errores, además de comprobar que el sistema no colapsa bajo el estrés de un número elevado de ciclos.

6.2 Experimentación y calibración de condiciones iniciales

El simulador se sometió a múltiples ejecuciones modificando las variables iniciales de configuración para analizar la sensibilidad del ecosistema global. Se parametrizaron escenarios asimétricos alterando los siguientes vectores:

- **Población:** pruebas con altas y bajas densidades iniciales para observar el desarrollo de poblaciones superpobladas y poblaciones emergentes.
- **Capital:** se evaluó cómo evolucionan las civilizaciones cuando el capital per cápita es elevado, y qué sucede si ese capital es mínimo.
- **Recursos (energía, tierra):** la reducción artificial de la disponibilidad de tierras y energía es una prueba necesaria para comprobar si una civilización puede o no sobrevivir con escasez de recursos. También es interesante comprobar qué sucede si esos recursos no escasean, sino que sobran.

Estas pruebas se realizaron mediante un estudio de qué conjuntos eran los que más se adaptaban a los resultados esperados. Se estableció una escala máxima a la que la velocidad de ejecución era tan lenta que hacía el sistema inviable (10^6 - 10^8), y se dividió el área útil en subpartes, que se fueron probando una a una. En un primer momento se intentó realizar una aproximación del modelo de *Optimización Bayesiana*, pero la complejidad para establecer un parámetro que midiese el rendimiento de la población hizo que se descartase esta opción.

[2] En la simulación no parece que tenga ninguna relevancia ya que no se llega a mostrar el avance de los ciclos, pero esta variable es necesaria para seguir el progreso de avance de las civilizaciones, y recargar la ventana

6.3 Ajuste de las fórmulas matemáticas

La parte más compleja de la fase de experimentación consistió en ajustar los coeficientes y las constantes de las ecuaciones para evitar dos comportamientos indeseados: el determinismo plano y el colapso prematuro. Los principales módulos que se ajustaron fueron el económico y el sociopolítico, ya que son los que más afectan al desarrollo de la civilización.

7. Declaración de IA y aspectos técnicos

7.1 Declaración de uso de Inteligencia Artificial

En el desarrollo de este proyecto se ha integrado el uso de modelos de IA (Gemini) como una herramienta de apoyo complementaria. Cuando se emplea de forma correcta, la IA resulta un recurso sumamente útil, no para sustituir el trabajo de desarrollo, sino para mejorar el aprendizaje y asegurar un código funcional. En este caso en particular, se ha utilizado principalmente en los siguientes puntos:

- **Comprensión de conceptos complejos:** ha servido como apoyo para asimilar la teoría de los sistemas concurrentes a bajo nivel, ayudando a entender y razonar por qué utilizar procesos independientes para esquivar el GIL (*Global Interpreter Lock*) de *Python*, o el funcionamiento de las barreras de sincronización y los cerrojos reentrantes.
- **Investigación de modelos matemáticos:** asistencia en la búsqueda, selección y comprensión de las funciones matemáticas y modelos que mejor se ajustan al proyecto, como la *Matriz de Leslie* o la función de producción de *Cobb-Douglas*.
- **Auditoría y optimización de código:** una vez probado el código final, y viendo que funciona, se ha empleado la IA para revisarlo en busca de errores de sintaxis, despistes lógicos o fugas de concurrencia que no hayan salido a la luz en las pruebas. Además, ha sido una herramienta muy útil a la hora de depurar el código, debido a que los errores *multiprocessing* no son nada intuitivos, y normalmente requieren de una investigación profunda en webs y foros especializados como *StackOverflow* para averiguar cuál es el problema.
- **Redacción de la memoria:** ha actuado como asistente para estructurar y dar un formato formal a las diferentes secciones de la documentación técnica.

7.2 Guía de mantenimiento

El código fuente se ha estructurado bajo principios de modularidad para permitir que el usuario pueda alterar algunas de las reglas del ecosistema global sin necesidad de reescribir la lógica de concurrencia. Las constantes de control se pueden modificar siguiendo las siguientes indicaciones:

- **Modificación de los ciclos de simulación:** el número de iteraciones que ejecuta la simulación se almacena en la variable *max_year*. Para ampliar o reducir el número de iteraciones, se debe modificar esta variable dentro de *simulacion/civilizacion.py*, en el *init* de la clase. Si el número de ciclos supera los 10_000, se debe modificar también la

[2] En la simulación no parece que tenga ninguna relevancia ya que no se llega a mostrar el avance de los ciclos, pero esta variable es necesaria para seguir el progreso de avance de las civilizaciones, y recargar la ventana

variable *max_year* dentro del archivo *gui/state.py* para que el *frontend* no sufra desajustes de renderizado^[2]

- **Alteración del tamaño de la caché de relaciones:** el número de ranuras de memoria destinadas a las relaciones de segunda oportunidad comerciales está limitado para forzar la asimetría epidemiológica. Si se desea incrementar el tamaño de la caché para permitir almacenar más datos, es necesario modificar el atributo *capacidad_cache* dentro de la inicialización de la clase *Civilización*. Esta modificación puede ser útil en caso de que se quieran lanzar más civilizaciones.
- **Número de civilizaciones a simular:** La arquitectura del motor de simulación es 100% escalable y permite añadir nuevos nodos de forma simétrica modificando la lista de control del orquestador. No obstante, la capa de presentación (GUI) requiere un acoplamiento intencionado, requiriendo la inserción de los nuevos *Checkboxes* de filtrado en el *Dashboard* y sus correspondientes formularios en la vista de parametrización.

7.3 Limitaciones técnicas y comportamiento inerte de la GUI

Durante las pruebas de estrés del simulador, se ha identificado una limitación técnica crítica relacionada con el renderizado reactivo de la librería gráfica *Flet* cuando coexiste con una infraestructura pesada de multiprocesamiento nativo.

- **El problema del bloqueo visual:** al lanzar la simulación, el hilo de la interfaz se pone a escuchar la cola de progreso. Bajo ciertas condiciones, el hilo de renderizado de *Flet* sufre una degradación de prioridad. El motor matemático sigue calculando en segundo plano y escribiendo los CSV con normalidad, pero la interfaz deja de actualizar sus widgets de forma fluida. Esto causa que la pantalla no se refresque, aunque la simulación haya terminado.
- **Solución de compromiso:** tras agotar las vías de resolución técnica, se ha determinado que este comportamiento responde a un cuello de botella interno del ciclo de eventos del motor de *Flet* al gestionar contextos asíncronos cruzados.
- **Procedimiento de reactivación:** Es importante destacar que este problema en la actualización de la interfaz no se presenta de forma sistemática; de manera ordinaria, el hilo de renderizado procesa los eventos correctamente y la aplicación transiciona de forma autónoma hacia la vista de resultados históricos. Para mitigar cualquier ambigüedad durante la evaluación, el usuario puede monitorizar el progreso en tiempo real a través de la terminal de comandos subyacente que despliegan los scripts .bat o .sh. En el instante en que la consola imprima el mensaje de finalización global (FIN DE LA SIMULACION), se constata que los hilos han concluido la persistencia de los datos en los ficheros CSV. Si tras la aparición de este mensaje la interfaz gráfica no actualiza automáticamente la pantalla de carga, el usuario simplemente debe forzar una actualización maximizando la ventana a pantalla completa y restaurándola a su tamaño original. Esta acción fuerza la transición hacia el panel de control y el *dashboard* interactivo.

[2] En la simulación no parece que tenga ninguna relevancia ya que no se llega a mostrar el avance de los ciclos, pero esta variable es necesaria para seguir el progreso de avance de las civilizaciones, y recargar la ventana

8. Conclusiones

Elegí este proyecto porque me llamaba mucho la atención y estaba interesado en ver cómo evolucionan una serie de entidades autónomas y sistemas dinámicos en función de los parámetros iniciales que les pongas, interactuando constantemente entre sí. Sinceramente, el desarrollo del simulador me ha parecido muy entretenido y retador. Al principio, lidiar con la arquitectura multiproceso y el paso de mensajes asíncronos fue un dolor de cabeza, sobre todo por lo poco intuitivos que son los errores de concurrencia, pero ver el motor funcionando en ráfaga a 1000 años y sin colgarse ha sido muy satisfactorio.

Lo que más me ha enganchado ha sido "*jugar*" con los parámetros en el archivo de configuración y en las propias fórmulas matemáticas. Es increíble ver cómo un cambio mínimo en los datos de partida de una sola civilización acaba afectando de forma indirecta a otra totalmente distinta por el motivo que sea. Ver cómo estas dinámicas complejas emergen de forma orgánica simplemente conectando procesos independientes a través de colas de comunicación ha hecho que el trabajo pase de ser una entrega obligatoria a un experimento interactivo genial.

9. Bibliografía

[1] Modelo World3: referencia teórica utilizada para la implementación de diferentes módulos, sirvió de *inspiración* para desarrollar algunas de las mecánicas clave.

Enlace: <https://github.com/Juji29/MyWorld3.git>

[2] Modelo SIR: utilizado para desarrollar el módulo de salud de forma realista.

Enlace:

<https://culturacientifica.com/2020/08/24/el-modelo-sir-un-enfoque-matematico-de-la-propagacion-de-infecciones/>

[3] Inteligencia Artificial: utilizada para la comprensión de conceptos complejos y optimización de estructuras.

Enlace: gemini.google.com